

Project 1: WAZUH SIEM Deployment & Configuration

Complete Implementation Guide

Table of Contents

1. [Project Overview](#)
 2. [Scope and Objectives](#)
 3. [Prerequisites](#)
 4. [Implementation Steps](#)
 5. [Testing and Validation](#)
 6. [Troubleshooting](#)
 7. [Results and Outcomes](#)
 8. [Conclusion](#)
 9. [References](#)
-

Project Overview

This document provides a comprehensive guide for deploying and configuring Wazuh SIEM (Security Information and Event Management) as part of a cybersecurity lab environment. Wazuh is an open-source security monitoring platform that combines SIEM capabilities, intrusion detection, file integrity monitoring, and compliance reporting.

What is Wazuh?

Wazuh is a comprehensive security platform that provides:

- **SIEM capabilities:** Log collection, analysis, and correlation
 - **Intrusion Detection System (IDS):** Real-time threat detection
 - **File Integrity Monitoring (FIM):** Detection of unauthorized file changes
 - **Compliance reporting:** Support for PCI DSS, HIPAA, NIST, GDPR standards
 - **Incident response:** Automated response capabilities
-

Scope and Objectives

Project Scope

This project focuses on:

- Deploying Wazuh manager on a dedicated virtual machine
- Installing and configuring Wazuh agents across six diverse systems:
 - MISP (Malware Information Sharing Platform) VM
 - ELK-LogPlatform (Log analysis system) VM
 - OpenCTI-ThreatIntel (Cyber threat intelligence platform) VM
 - Shuffle-SOAR (Security orchestration platform) VM
 - Windows Server DC (Enterprise Windows infrastructure)
 - DVWA WebServer (Web application security platform)
- Integrating network infrastructure via syslog forwarding:
 - pfSense firewall systems (Mac Mini and VM)
 - Metasploitable vulnerable system
- Establishing secure communication between manager and all agents/sources
- Configuring comprehensive log collection across Linux, Windows, and network infrastructure
- Testing event generation and detection capabilities across heterogeneous platforms
- Validating dashboard functionality and cross-platform event visualization

Objectives

Primary Objectives:

- Deploy a functional Wazuh SIEM environment
- Establish agent-to-manager communication
- Configure basic security monitoring
- Validate log collection and event processing

Learning Outcomes:

- Understanding SIEM deployment architecture
- Hands-on experience with log collection and parsing
- Security rule development fundamentals
- Incident detection workflow implementation
- Dashboard navigation and event analysis

Prerequisites

Infrastructure Requirements

Minimum System Requirements:

- **wazuh-manager.service - Wazuh manager**
Loaded: loaded (/lib/systemd/system/wazuh-manager.service; enabled; vendor preset: enabled)
Active: active (running) since Mon 2025-09-23 10:05:34 UTC; 2h 37min ago

Step 1.2: Verify Wazuh API Status

```
bash

# Check Wazuh API status (integrated in newer versions)
sudo systemctl status wazuh-api
```

Note: In Wazuh 4.3+, the API is integrated into the manager. If service not found, this is expected behavior.

Step 1.3: Check Dashboard Service

```
bash

# Verify dashboard service
sudo systemctl status wazuh-dashboard
```

Step 1.4: Verify Installation Components

```
bash

# Check Wazuh manager installation directory
ls -la /var/ossec/

# Check configuration files
ls -la /var/ossec/etc/

# Check indexer status (for all-in-one installations)
sudo systemctl status wazuh-indexer
```

Step 1.5: Network Connectivity Verification

```
bash

# Check listening ports for Wazuh services
sudo netstat -tlnp | grep -E '1514|1515|55000|443'

# Check firewall status
sudo ufw status

# For CentOS/RHEL systems:
sudo firewall-cmd --list-all
```

Expected Output:

```
tcp      0      0 0.0.0.0:55000      0.0.0.0:*        LISTEN   2635/python3
tcp6    0      0 :::55000            :::*              LISTEN   2635/python3
```

Phase 2: Manager Service Troubleshooting

If the Wazuh manager is not running, start it using the following commands:

Step 2.1: Start Wazuh Manager

```
bash

# Start the Wazuh manager service
sudo systemctl start wazuh-manager

# Verify the service started successfully
sudo systemctl status wazuh-manager
```

Step 2.2: Check for Startup Issues

```
bash

# View service logs if startup fails
sudo journalctl -u wazuh-manager -f

# Check manager logs
sudo tail -20 /var/ossec/logs/ossec.log
```

Phase 3: Web Interface Access

Step 3.1: Access Dashboard

Access the Wazuh dashboard through a web browser:

- **URL:** `https://192.168.30.10:443`
- **Default Credentials:**
 - Username: `admin`
 - Password: `admin` (or check installation files)

Step 3.2: Alternative Credential Recovery

If default credentials don't work, try these methods:

```
bash
```

```
# Search for password files
```

```
sudo find /root -name "*password*" -type f 2>/dev/null
```

```
# Check installation logs
```

```
sudo grep -i "password" /var/log/wazuh-indexer/wazuh-indexer.log 2>/dev/null
```

```
# Look for Wazuh installation files
```

```
sudo tar -axf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt -O 2>/dev/null
```

Phase 4: Agent Deployment

Step 4.1: Agent Installation Commands Generation

In the Wazuh dashboard:

1. Navigate to **Agents** → **Deploy new agent**

2. Configure agent parameters:

- **Operating System:** Linux (Ubuntu)
- **Architecture:** x64
- **Wazuh server address:** 192.168.30.10
- **Agent name:** MISP-ThreatIntel

Step 4.2: Pre-installation Checks on MISP VM

```
bash
```

```
# Test connectivity to Wazuh manager
```

```
ping -c 3 192.168.30.10
```

```
# Check system information
```

```
uname -a
```

```
# Verify required tools are available
```

```
which curl wget
```

```
# Update system packages
```

```
sudo apt update
```

Step 4.3: Agent Installation on MISP VM

The dashboard generates installation commands similar to:

```
bash
```

```
# Download and install Wazuh agent
```

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.x.x-1_amd64.deb && sudo V
```

```
# Enable and start the agent service
```

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable wazuh-agent
```

```
sudo systemctl start wazuh-agent
```

Step 4.4: Verify Agent Installation

```
bash
```

```
# Check agent service status
```

```
sudo systemctl status wazuh-agent
```

```
# View agent logs
```

```
sudo tail -20 /var/ossec/logs/ossec.log
```

Phase 5: Agent-Manager Communication Verification

Step 5.1: Verify Agent Registration (Manager Side)

On the Wazuh Manager VM:

```
bash
```

```
# List all registered agents
```

```
sudo /var/ossec/bin/agent_control -lc
```

```
# Check manager logs for agent connections
```

```
sudo tail -20 /var/ossec/logs/ossec.log | grep MISP
```

Expected Output:

```
Wazuh agent_control. List of available agents:
```

```
ID: 000, Name: wazuh-server (server), IP: 127.0.0.1, Active/Local
```

```
ID: 001, Name: MISP-ThreatIntel, IP: 192.168.30.50, Active
```

Step 5.2: Dashboard Verification

In the Wazuh dashboard:

1. Navigate to **Agents** section
2. Verify "MISP-ThreatIntel" appears as **Active**

3. Confirm agent count shows "Active (1)"

Phase 6: Configuration Troubleshooting

Step 6.1: Initial Configuration Assessment

Check the default agent configuration:

```
bash

# View current agent configuration
sudo cat /var/ossec/etc/ossec.conf | grep -A 10 "<localfile>"
```

Initial Issue: If no output is shown, the agent lacks log source configuration.

Step 6.2: Configuration File Analysis

View the complete configuration structure:

```
bash

# Display full configuration file
sudo cat /var/ossec/etc/ossec.conf
```

The standard configuration includes:

- Client connection settings
- Log file monitoring (localfile entries)
- File integrity monitoring (syscheck)
- System inventory collection
- Security configuration assessment

Step 6.3: Avoiding Configuration Errors

Critical Warning: When modifying configuration files, always:

1. Create backups before editing
2. Validate XML syntax
3. Test changes incrementally
4. Monitor service logs after modifications

```
bash
```

```
# Create configuration backup
sudo cp /var/ossec/etc/ossec.conf /var/ossec/etc/ossec.conf.backup

# Restore backup if needed
sudo cp /var/ossec/etc/ossec.conf.backup /var/ossec/etc/ossec.conf
```

Phase 9: Multi-Agent Deployment - ELK VM

After successfully deploying the first agent on MISP VM, expanding to additional systems provides broader security coverage and demonstrates scalability.

Step 9.1: ELK Agent Deployment Preparation

Power on the ELK VM and verify network connectivity:

```
bash

# Test connectivity to Wazuh manager
ping -c 3 192.168.30.10

# Verify system information
uname -a

# Check available tools
which curl wget

# Update system packages
sudo apt update
```

Step 9.2: Generate ELK Agent Installation Commands

In the Wazuh dashboard:

1. Navigate to **Agents** → **Deploy new agent**
2. Configure agent parameters:
 - **Operating System:** Linux (Ubuntu/Debian)
 - **Architecture:** x64
 - **Wazuh server address:** 192.168.30.10
 - **Agent name:** ELK-LogPlatform

Step 9.3: Install ELK Agent

Execute the dashboard-generated installation commands:

```
bash
```

```
# Download and install Wazuh agent (use exact commands from dashboard)
```

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.x.x-1_amd64.deb && sudo V
```

```
# Enable and start the agent service
```

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable wazuh-agent
```

```
sudo systemctl start wazuh-agent
```

Step 9.4: Verify ELK Agent Installation

```
bash
```

```
# Check agent service status
```

```
sudo systemctl status wazuh-agent
```

```
# View agent logs for connection status
```

```
sudo tail -20 /var/ossec/logs/ossec.log
```

Phase 10: OpenCTI Agent Deployment

Expanding to the third agent system further demonstrates the scalability of the SIEM architecture.

Step 10.1: OpenCTI Agent Installation

Following the same deployment pattern:

```
bash
```

```
# Generate agent from Wazuh dashboard with agent name: OpenCTI-ThreatIntel
```

```
# Install using dashboard-generated commands
```

```
sudo systemctl status wazuh-agent # Verify active status
```

Phase 11: Shuffle-SOAR Agent Deployment

Adding security orchestration platform monitoring:

```
bash
```

```
# Deploy agent with name: Shuffle-SOAR
```

```
# Verify installation and connectivity
```

```
sudo systemctl status wazuh-agent # Confirm active status
```

Phase 12: Windows Server Agent Deployment

Cross-platform monitoring with Windows Server integration:

Step 12.1: Windows Agent Installation

In PowerShell as Administrator:

```
powershell

# Test connectivity to Wazuh manager
Test-NetConnection -ComputerName 192.168.30.10 -Port 1514

# Install Windows agent using dashboard-generated MSI commands
# Verify service status
Get-Service -Name "WazuhSvc"
```

Step 12.2: Windows Agent Troubleshooting

If agent shows "pending" status:

```
powershell

# Stop and restart the service
NET STOP WazuhSvc
NET START WazuhSvc

# Check client keys
Get-Content "C:\Program Files (x86)\ossec-agent\client.keys"
```

Phase 13: DVWA Web Server Agent Deployment

Web application security monitoring integration:

Step 13.1: DVWA Agent Installation

```
bash

# Deploy agent with name: DVWA-WebServer
# Configure Apache log monitoring (already present in default config)
sudo systemctl status wazuh-agent # Verify active status
```

Step 13.2: Apache Log Monitoring Configuration

The DVWA agent includes pre-configured Apache monitoring:

```
bash
```

```
# Verify Apache log configuration
sudo cat /var/ossec/etc/ossec.conf | grep -A 5 apache

# Apache logs monitored:
# - /var/log/apache2/access.log
# - /var/log/apache2/error.log
```

Phase 14: pfSense Network Infrastructure Integration

Network infrastructure monitoring via syslog forwarding:

Step 14.1: Configure Wazuh Manager for Syslog Reception

```
bash

sudo nano /var/ossec/etc/ossec.conf

# Add syslog configuration:
<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips>192.168.10.0/24</allowed-ips>
  <allowed-ips>192.168.2.0/24</allowed-ips>
  <allowed-ips>192.168.30.0/24</allowed-ips>
  <allowed-ips>192.168.40.0/24</allowed-ips>
</remote>

# Restart manager
sudo systemctl restart wazuh-manager
```

Step 14.2: Verify Syslog Listener

```
bash

# Confirm syslog reception active
sudo netstat -ulnp | grep 514
# Expected: UDP port 514 listening
```

Step 14.3: Configure pfSense Remote Logging

In pfSense web interface:

1. Navigate to **Status** → **System Logs** → **Settings**
2. Configure Remote Logging:
 - **Remote Log Server:** 192.168.30.10:514

- **Enable Remote Logging:** Checked
- **Log Contents:** Firewall, System, Authentication events only

Step 14.4: Verify pfSense Integration

```
bash

# Monitor for pfSense syslog messages
sudo tail -f /var/ossec/logs/ossec.log

# Search for pfSense entries
sudo grep -i "192.168" /var/ossec/logs/ossec.log | tail -10
```

Phase 15: Metasploitable Syslog Integration

Vulnerable system monitoring provides visibility into potential attack activities and system exploitation attempts.

Step 15.1: Configure Metasploitable Syslog Forwarding

Check the syslog system on Metasploitable:

```
bash

# Identify running syslog daemon
ps aux | grep syslog

# Check for configuration files
ls -la /etc/syslog*
ls -la /etc/rsyslog*
```

Step 15.2: Add Remote Logging Configuration

Configure syslog forwarding to Wazuh manager:

```
bash
```

```
# For traditional syslogd
sudo nano /etc/syslog.conf
# Add: *.* @192.168.30.10:514

# For rsyslog (more common)
sudo nano /etc/rsyslog.conf
# Add: *.* @192.168.30.10:514

# Restart syslog service
sudo /etc/init.d/syslogd restart
```

Step 15.3: Test Metasploitable Integration

```
bash

# Generate test log events
logger "Metasploitable SIEM integration test"

# Verify log forwarding on Wazuh Manager
sudo grep -i "metasploitable" /var/ossec/logs/ossec.log | tail -10
```

Phase 16: Complete Multi-Source Registration Verification

Step 16.1: Final Agent and Source Status Check

On the Wazuh Manager VM:

```
bash

# List all registered agents and sources
sudo /var/ossec/bin/agent_control -lc
```

Expected Nine-Source Output:

```
Wazuh agent_control. List of available agents:
ID: 000, Name: wazuh-server (server), IP: 127.0.0.1, Active/Local
ID: 001, Name: MISP-ThreatIntel, IP: any, Active
ID: 002, Name: OpenCTI-ThreatIntel, IP: any, Active
ID: 003, Name: Shuffle-SOAR, IP: any, Active
ID: 004, Name: Windows-Server-DC, IP: any, Active
ID: 005, Name: ELK-LogPlatform, IP: any, Active
ID: 006, Name: DVWA-WebServer, IP: any, Active
+ pfSense Mac Mini syslog source (network infrastructure)
+ pfSense VM syslog source (network management)
+ Metasploitable syslog source (vulnerable system monitoring)
```

Testing and Validation

Phase 7: Event Generation Testing

Step 7.1: Generate Authentication Events

On the MISP VM:

```
bash

# Generate sudo authentication events
sudo su - root
exit

# Generate additional system events
echo "wazuh test file" > /tmp/test-file.txt
rm /tmp/test-file.txt

# Create network activity
ping -c 3 google.com
```

Step 7.2: Verify Log Transmission

```
bash

# Check agent logs for data transmission
sudo tail -10 /var/ossec/logs/ossec.log | grep -i "sending\|info\|connected"

# Monitor real-time log activity
sudo tail -f /var/ossec/logs/ossec.log
```

Phase 8: Dashboard Event Analysis

Step 8.1: Navigate to Security Events

In the Wazuh dashboard:

1. Click **"Explore"** in the left sidebar
2. Access **"Events"** or **"Discover"** section
3. Set time filter to "Last 15 minutes"
4. Filter by agent:

Step 8.2: Event Analysis

The dashboard should display events with the following structure:

- **Timestamp:** Event occurrence time
 - **Agent Information:** Name, IP address, ID
 - **Event Details:** Source, rule information, compliance mappings
 - **Rule Classification:** Level, description, MITRE ATT&CK techniques
-

Troubleshooting

Common Issues and Solutions

Issue 1: Agent Service Fails to Start

Symptoms:

- `systemctl status wazuh-agent` shows "failed" or "inactive (dead)"
- Error messages in system logs

Solution:

```
bash

# Check detailed error logs
sudo journalctl -u wazuh-agent -n 20

# Verify configuration syntax
sudo /var/ossec/bin/wazuh-control status

# Common fix: restart after configuration changes
sudo systemctl restart wazuh-agent
```

Issue 2: No Events Appearing in Dashboard

Symptoms:

- Agent shows as "Active" but no events in dashboard
- Empty event logs

Troubleshooting Steps:

```
bash
```

```
# Verify agent is sending data
sudo grep -i "sending" /var/ossec/logs/ossec.log | tail -5

# Check if logs are being monitored
sudo cat /var/ossec/etc/ossec.conf | grep -A 5 "<localfile>"

# Verify time synchronization
timedatectl status
```

Issue 3: Configuration File Errors

Symptoms:

- Service fails to start after configuration changes
- XML parsing errors in logs

Solution:

```
bash

# Restore working configuration
sudo cp /var/ossec/etc/ossec.conf.backup /var/ossec/etc/ossec.conf

# Validate XML syntax before applying changes
xmllint --noout /var/ossec/etc/ossec.conf
```

Results and Outcomes

Project Success Metrics

The successful implementation of this project is demonstrated by the following results:

Functional Verification

System Status Confirmation:

- Wazuh Manager: **Active (Running)**
- Wazuh Dashboard: **Accessible** at <https://192.168.30.10:443>
- Agent Communication: **MISP-ThreatIntel** shows as **Active**
- API Service: **Port 55000** listening and responsive

Security Event Collection Evidence

The following sample events demonstrate successful log collection and processing from multiple agents and sources:

MISP-ThreatIntel Agent Events:

Time: Sep 23, 2025 @ 12:42:51.371

Agent: MISP-ThreatIntel (192.168.30.50)

Rule ID: 5501 - PAM: Login session opened

MITRE Technique: T1078 (Valid Accounts)

Event: sudo[16199]: pam_unix(sudo:session): session opened for user root(uid=0) by misp(uid=1000)

Time: Sep 23, 2025 @ 12:42:51.323

Agent: MISP-ThreatIntel (192.168.30.50)

Rule ID: 5402 - Successful sudo to ROOT executed

MITRE Technique: T1548.003 (Sudo and Sudo Caching)

Event: sudo[16199]: misp : TTY=tty1 ; PWD=/home/misp ; USER=root ; COMMAND=/usr/bin/tail -10 /var/ossec/logs/ossec.log

ELK-LogPlatform Agent Events:

Time: Sep 23, 2025 @ 14:29:49.840

Agent: ELK-LogPlatform (192.168.30.20)

Rule ID: 40704 - Systemd: Service exited due to a failure

Rule Level: 5 (Medium severity)

Event: logstash.service: Main process exited, code=exited, status=1/FAILURE

Time: Sep 23, 2025 @ 14:29:29.196

Agent: ELK-LogPlatform (192.168.30.20)

Rule ID: 5502 - PAM: Login session closed

MITRE Technique: T1078 (Valid Accounts)

Event: pam_unix(sudo:session): session closed for user root

OpenCTI-ThreatIntel Agent Events:

Time: Sep 23, 2025 @ 20:46:22.133

Agent: OpenCTI-ThreatIntel (192.168.30.60)

Rule ID: 52002 - AppArmor DENIED

Rule Level: 3 (Low severity)

Event: kernel: audit: type=1400 audit(1758654471.072:336): apparmor="DENIED" operation="open"

Windows-Server-DC Agent Events:

Time: Sep 23, 2025 @ 21:56:51.105

Agent: Windows-Server-DC (192.168.30.40)

Rule Level: 3 (Informational)

Event Type: Windows Event Log - .NET Framework Updates

Event: 2025-07 Cumulative Update for .NET Framework 3.5, 4.8 and 4.8.1 for Microsoft server operating system version 21H2

DVWA-WebServer Agent Events:

Time: Sep 24, 2025 @ 09:54:02.076

Agent: DVWA-WebServer (192.168.10.20)

Rule ID: 5502 - PAM: Login session closed

MITRE Technique: T1078 (Valid Accounts)

Event: pam_unix(sudo:session): session closed for user root

pfSense Syslog Integration:

Time: Sep 24, 2025 @ 09:08:22

Source: Remote syslog (pfSense Mac Mini - 192.168.10.x networks)

Event Type: Network infrastructure logs - firewall rules, system events

Source: Remote syslog (pfSense VM - 192.168.10.5)

Event Type: Network management logs - internal traffic monitoring

Integration: Firewall rules, authentication, system events

Metasploitable Syslog Integration:

Time: Sep 24, 2025 @ 13:42:15

Source: Remote syslog (192.168.30.10)

Event Type: Vulnerable system monitoring

Event: Traditional syslogd messages from intentionally vulnerable platform

Purpose: Attack pattern visibility and exploitation detection

Compliance and Security Framework Integration

Events are automatically mapped to multiple compliance frameworks:

- **PCI DSS:** 10.2.5, 10.2.2 (Audit logs and privileged user access)
- **HIPAA:** 164.312.b (Information access management)
- **NIST 800-53:** AU.14, AC.7 (Audit review and unsuccessful login attempts)
- **GDPR:** IV_32.2, IV_35.7.d (Security of processing, security measures)

Event Categories Successfully Monitored

1. Authentication Events

- Successful/failed login attempts
- PAM session management
- Privilege escalation activities

2. System Administration

- Sudo command execution
- Service management operations
- System configuration changes

3. Agent Management

- Agent startup/shutdown events
- Connection status monitoring
- Health check reporting

Key Performance Indicators

Deployment Metrics:

- **Setup Time:** ~8-10 hours including complete nine-source deployment and troubleshooting
- **Agent Deployment Success Rate:** 100% (6/6 agents successfully deployed)
- **Syslog Integration Success:** 100% (3/3 syslog sources active - dual pfSense + Metasploitable)
- **Event Processing Latency:** <5 minutes from event generation to dashboard visibility
- **System Resource Usage:** Minimal impact on VM performance across all systems

Security Coverage:

- **Monitored Systems:**
 - 5 Linux VMs: MISP-ThreatIntel (192.168.30.50), OpenCTI-ThreatIntel (192.168.30.60), ELK-LogPlatform (192.168.30.20), Shuffle-SOAR (192.168.30.70), DVWA-WebServer (192.168.10.20)
 - 1 Windows System: Windows-Server-DC (192.168.30.40)
 - 3 Syslog Sources: pfSense Mac Mini (192.168.10.x), pfSense VM (192.168.10.5), Metasploitable (192.168.30.10)
- **Network Coverage:** Four network segments (192.168.30.x, 192.168.10.x, 192.168.2.x, 192.168.40.x)
- **Log Sources:** journald, authentication logs, system logs, systemd services, Windows Event Logs, Apache access/error logs, firewall logs, vulnerable system logs
- **Rule Coverage:** 25+ built-in detection rules activated across platforms
- **Compliance Frameworks:** 4 major frameworks (PCI DSS, HIPAA, NIST, GDPR)
- **MITRE ATT&CK Mapping:** Techniques T1078, T1548.003, T1562.001, T1499 identified

Enterprise Architecture Benefits:

- **Hybrid Integration:** Agent-based monitoring + syslog forwarding approach
 - **Multi-Platform Coverage:** Linux, Windows, and network infrastructure visibility
 - **Vulnerable System Monitoring:** Attack pattern detection through Metasploitable integration
 - **Network Infrastructure Redundancy:** Dual pfSense monitoring for comprehensive network visibility
 - **Cross-System Event Correlation:** Nine-source event correlation capabilities
 - **Production Scalability:** Demonstrated enterprise-grade architecture supporting heterogeneous environments
-

Conclusion

Project Summary

This implementation successfully demonstrates the deployment of a functional Wazuh SIEM environment capable of collecting, processing, and analyzing security events from multiple sources. The project established a solid foundation for enterprise-level security monitoring with the following achievements:

Technical Accomplishments:

- Deployed Wazuh manager with full dashboard functionality and syslog reception capabilities
- Successfully configured multi-platform agent communication architecture (Linux + Windows)
- Established comprehensive log collection from heterogeneous systems across multiple network segments
- Implemented dual integration approach: agent-based monitoring + syslog forwarding for network infrastructure
- Validated cross-platform event processing and analysis across distributed infrastructure
- Demonstrated enterprise-grade compliance framework integration and MITRE ATT&CK mapping
- Achieved network infrastructure monitoring through pfSense syslog integration
- Implemented web application security monitoring with Apache log collection
- Established cross-system event correlation and centralized security visibility across seven distinct sources
- Proven scalability from single-agent to comprehensive enterprise architecture

Learning Outcomes Achieved:

- Hands-on experience with open-source SIEM deployment

- Understanding of agent-based monitoring architecture
- Practical knowledge of security event correlation
- Exposure to compliance reporting and MITRE ATT&CK framework integration

Skills & Career Relevance

This project demonstrates competencies directly aligned with cybersecurity career progression and industry requirements:

Technical Skills Developed:

SIEM Architecture & Deployment

- Multi-platform SIEM deployment (Linux + Windows agents, syslog integration)
- Network security monitoring through firewall log integration (dual pfSense setup)
- Cross-platform event correlation and analysis across heterogeneous environments
- Real-time threat detection and incident validation workflows

Security Operations & Analysis

- Log analysis across diverse platforms (Linux, Windows, network infrastructure, web applications, vulnerable systems)
- Security rule tuning and compliance alignment (PCI DSS, HIPAA, GDPR, NIST 800-53)
- MITRE ATT&CK framework mapping and threat technique identification
- Event investigation and forensic analysis methodologies

Integration & Automation

- SOAR platform exposure through Shuffle integration concepts
- Threat intelligence platform integration (MISP, OpenCTI)
- Web application security monitoring with Apache log analysis
- Network device integration through syslog forwarding protocols
- Vulnerable system monitoring for security testing and attack pattern analysis

Professional Competencies:

- Complex multi-system troubleshooting in virtualized environments
- Technical documentation and professional reporting
- Architecture design with demonstrated scalability from single-agent to enterprise deployment
- Project management through systematic implementation phases

Career Path Alignment:

SOC Analyst (Entry-Level)

- Log analysis and event correlation across multiple platforms
- MITRE ATT&CK technique mapping and threat categorization
- Alert validation and incident escalation procedures
- Compliance framework understanding for audit support

Security Engineer (Mid-Level)

- SIEM deployment and agent management across enterprise infrastructure
- Network security device integration and log forwarding configuration
- Cross-platform security monitoring architecture design
- Security tool integration and workflow automation

Incident Response Specialist

- Multi-source event correlation for incident reconstruction
- Forensic log analysis across Windows, Linux, and network devices
- Threat hunting methodologies using centralized security monitoring
- Automated response integration through SOAR platform concepts

Cybersecurity Consultant (Senior-Level)

- Open-source security solution evaluation and implementation
- Compliance framework reporting and audit preparation
- Enterprise architecture design for heterogeneous environments
- Cost-benefit analysis for security technology investments

Skills Progression Roadmap:

- **Entry Level:** Hands-on SIEM operation, log analysis, basic event correlation
- **Mid Level:** Multi-platform deployment, custom rule development, SOAR integration
- **Senior Level:** Enterprise SOC architecture, cloud integration, advanced threat analytics

Lessons Learned

Configuration Management:

- Always backup configuration files before modifications
- Validate XML syntax to prevent service failures
- Implement incremental changes with testing at each step

Troubleshooting Best Practices:

- Monitor service logs during configuration changes
- Verify network connectivity before agent deployment
- Use built-in diagnostic tools for systematic problem resolution

Future Enhancements

Phase 2: Immediate Next Steps (0-3 months)

1. Advanced Rule Development:

- Develop correlation rules to detect brute force attacks across Windows Server DC authentication logs and pfSense firewall blocks
- Create custom detection signatures for web application attacks in DVWA Apache logs
- Implement behavioral analytics rules for privilege escalation patterns across Linux agents

2. SOAR Integration Enhancement:

- Configure Shuffle-SOAR automated playbooks for incident response workflows
- Implement automated account lockout responses for detected brute force attempts
- Develop automated threat hunting queries triggered by OpenCTI indicator updates

3. Email Alerting & Notification:

- Configure SMTP integration for critical security events (Rule Level 10+)
- Implement alert escalation workflows based on event severity and compliance requirements
- Set up executive dashboard reports for security posture visibility

Phase 3: Intermediate Expansion (3-6 months)

1. Threat Intelligence Integration:

- Extend OpenCTI integration for automated IoC enrichment of Wazuh alerts
- Configure MISP feed integration for malware signature updates
- Implement threat hunting dashboards combining internal events with external threat intel

2. Cloud Infrastructure Expansion:

- Integrate AWS CloudTrail logs via syslog forwarding or agent deployment
- Add Azure Sentinel log forwarding for hybrid cloud visibility
- Implement container security monitoring for Docker/Kubernetes environments

3. Advanced Analytics:

- Deploy machine learning models for anomaly detection in user behavior
- Implement statistical baseline monitoring for network traffic patterns
- Configure automated threat scoring based on MITRE ATT&CK technique correlation

Phase 4: Long-term Enterprise Roadmap (6+ months)

1. Full SOC Automation:

- Complete SOAR integration with automated incident classification and response
- Implement DevSecOps CI/CD pipeline integration for SIEM configuration management
- Deploy security orchestration across entire security tool stack

2. Compliance & Governance:

- Automated compliance reporting for SOX, ISO27001, and regulatory frameworks
- Security metrics dashboard with KPI tracking for executive reporting
- Risk scoring integration with business impact analysis

3. Advanced Threat Detection:

- Implementation of User and Entity Behavior Analytics (UEBA)
- Integration with deception technology and honeypot networks
- Advanced persistent threat (APT) detection using cross-platform correlation

Value Proposition

This project establishes a production-ready SIEM environment that provides:

- **Cost-effective** security monitoring (open-source solution)
- **Comprehensive** visibility across heterogeneous infrastructure (Linux, Windows, network devices, vulnerable systems)
- **Multi-platform** event correlation and analysis capabilities
- **Compliance-ready** reporting across major frameworks (PCI DSS, HIPAA, NIST, GDPR)
- **Scalable** architecture demonstrated from single-agent to enterprise-level deployment
- **Integration-flexible** platform supporting both agent-based and syslog-based monitoring
- **Network infrastructure** visibility through dual firewall monitoring (pfSense Mac Mini + VM)
- **Vulnerable system** monitoring for attack pattern detection and security testing visibility

The successful implementation demonstrates the viability of open-source security solutions for organizations seeking robust SIEM capabilities without the licensing costs associated with commercial alternatives. The nine-source architecture provides comprehensive coverage across threat intelligence platforms, security orchestration tools, web applications, enterprise infrastructure, network devices, and vulnerable systems - representing a complete enterprise security monitoring solution with both production and security testing capabilities.

References

Documentation Resources

1. Official Wazuh Documentation: <https://documentation.wazuh.com/>
2. Wazuh Installation Guide: <https://documentation.wazuh.com/current/installation-guide/>
3. Agent Configuration Reference: <https://documentation.wazuh.com/current/user-manual/reference/ossec-conf/>
4. Rule Creation Guide: <https://documentation.wazuh.com/current/user-manual/ruleset/rules-classification.html>

Technical Standards

1. MITRE ATT&CK Framework: <https://attack.mitre.org/>
2. NIST Cybersecurity Framework: <https://www.nist.gov/cyberframework>
3. PCI DSS Requirements: <https://www.pcisecuritystandards.org/>
4. GDPR Compliance Guidelines: <https://gdpr-info.eu/>

Community Resources

1. Wazuh GitHub Repository: <https://github.com/wazuh/wazuh>
2. Community Forum: <https://wazuh.com/community/>
3. Integration Examples: <https://github.com/wazuh/wazuh-ansible>
4. Rule Contributions: <https://github.com/wazuh/wazuh-ruleset>

Document Version: 1.0

Last Updated: September 24, 2025

Author: Prageeth Panicker

Status: Production Ready

This document serves as both an implementation guide and a reference for cybersecurity professionals seeking to deploy open-source SIEM solutions. The methodologies and configurations presented have been tested in a controlled lab environment and can be adapted for production deployments with appropriate security hardening measures.